YCMMS Tam Sistem Mimarisi

Yönetici, Bakım, Üretim ve Planlama Sistemi Kapsamlı Dokümantasyonu

Tarih: 6 Mart 2026
Versiyon: 1.0
Sistem Tipi: React Native + Firebase
Dil: Türkçe

İçindekiler

1. Sistem Genel Yapısı
2. Modül 1: ÜRETİM (Arıza Bildirimi)
3. Modül 2: BAKIM (Bakım Yönetimi)
4. Modül 3: HAFTALIK BAKIM (Planlı Haftalık Bakım)
5. Modül 4: YILLIK BAKIM (Planlı Yıllık Bakım)
6. Modül 5: PROJE (Proje Planlama)
7. Modül 6: PLANLAMA (Operasyonel Planlama)
8. Firestore Veri Mimarisi
9. Multi-Tenant Mimarisi
10. Demo Mode Akışı

Sistem Genel Yapısı

Teknoloji Stack

- **Frontend:** React Native
- **Veritabanı:** Firebase Firestore
- **Kimlik Doğrulama:** Firebase Authentication
- **Tarih/Saat:** @react-native-community/datetimepicker
- **Seçici Bileşenler:** @react-native-picker/picker

Uygulama Akış Şeması

```
Login Ekranı
   ↓
Menu Ekranı ──────────────────────────────────────────────┐
   ├─ ÜRETİM (Arıza Bildirimi)                            │
   ├─ BAKIM (Bakım Menüsü)                                │
   │  ├─ Açık Bildirimler (Aktif Arızalar)               |
   │  ├─ Geçmiş Arızalar                                 │
   │  ├─ Haftalık Bakım                                  │
   │  ├─ Yıllık Bakım                                    │
   │  ├─ Bina Bakım                                      │
   │  └─ Yardımcı İşletmeler                             │
   ├─ PROJE (Proje Yönetimi)                              │
   │  ├─ Yapılacak Projeler                              │
   │  ├─ Tamamlanan Projeler                             │
   │  └─ Onay Bekleyen Projeler                          │
   └─ PLANLAMA (İş Planlama)                              |
```

```
      ├─ Haftalık Bakım Planlama                                  │
      ├─ Yıllık Bakım Planlama                                    │
      ├─ Proje Planlama                                           │
      ├─ Ayarlar (Makine, İsim, Vardiya, Neden)                   │
      └─ Veri Girişi                                              │
      ↑──────────────────────────────────────────────────────────
```

Kimlik Doğrulama & Kullanıcı Yönetimi

İşlem	Detaylar	Firebase
Giriş (Gerçek)	Email + Şifre	`signInWithEmailAndPassword()`
Demo Giriş	isDemoMode = true	Local state only
Şifre Sıfırlama	Email gönderilir	`sendPasswordResetEmail()`
Çıkış	Tüm state temizle	`signOut()`
Hesap Değişt	Çıkış + Yeni Giriş	Logout + Login screen
İşletme ID	Kullanıcıya atanmış	Multi-tenant isolation

Modül 1: ÜRETİM (Arıza Bildirimi)

Amaç

Üretim sırasında meydana gelen arıza/yıkılmış durumları bildirir ve izler.

Ekran: `uretim`

Bileşen	Tip	Kullanım	Durum
Başlangıç Tarihi	DatePicker	DD.MM.YYYY	Sistem tarihi
Başlangıç Saati	Picker (24h, 5' aralık)	HH:MM	Sistem saati
Makine Seç	Picker (makineler[])	Dropdown	Zorunlu
Sipariş No	TextInput	String	Opsiyonel
Açıklama	TextMultiInput	String	Zorunlu
BİLDİR BUTON	**SUBMIT**	**-> Firestore**	**İşlem**

Veri Akışı: Arıza Bildirme

```
Frontend (uretim ekranı)
   ↓
veriyiKaydet() fonksiyonu
   ├─ Doğrula: selectedMakine + tarihBaslangic + saatBaslangic
   ├─ Oluştur arızaObj = {
   │  işletme_id, makine_adi, siparis_no, aciklama,
   │  tarih_baslangic, saat_baslangic, status: 'bildirildi'
```

```
   │  kayit_tarihi: serverTimestamp()
   │}
   ├─ Demo Mode MI?
   │  ├─ EVET: State'e ekle (setArizalar), activeArizaCount arttır
   │  └─ HAYIR: Firestore'a addDoc('duruslar', arızaObj)
   ├─ Frontend: Formu sıfırla
   ├─ Refresh: arizalariYukle(işletmeId)
   └─ Navigate: setCurrentScreen('menu')
```

Firestore Koleksiyonu: `duruslar`

Alan	Tip	Açıklama	Örnek
işletme_id	String	Multi-tenant key	"isletme_123"
makine_adi	String	Makinenin adı	"Tornalama Tezgahı A1"
siparis_no	String	Üretim siparişi no	"SIP-2026-001"
aciklama	String	Arıza açıklaması	"Motor titremesi var"
tarih_baslangic	String	DD.MM.YYYY	"06.03.2026"
saat_baslangic	String	HH:MM	"14:30"
status	String	'bildirildi' | 'tamamlandi'	"bildirildi"
Bakım Alanları (opsiyonel)	-	Sağlayıcı bakım sırasında doldur	-
bakim_vardiya	String	Vardiya türü	"Sabah"
bakim_teknisyen	String	Bakım personeli	"Ahmet Y."
bakim_neden	String	Arıza nedeni	"Elektrik"
bakim_aciklama	String	Bakım detayları	"Kontakt değiştirildi"
mudahale_baslangic	String	HH:MM	"14:35"
tarih_bitis	String	DD.MM.YYYY	"06.03.2026"
saat_bitis	String	HH:MM	"15:45"
response_time	Integer	Dakika	15
kayit_tarihi	Timestamp	Sunucu tarafından	Otomatik

Durum Akış Şeması: ÜRETİM Arızası

```
[Bildirildi] (Ar ıza rapor edildi)
        ↓
[Müdahale] (Bakım personeli çalışıyor)
        ↓
[Tamamlandı] (Arıza çözüldü, status = 'tamamlandi')
```

İlişkili Fonksiyonlar

Fonksiyon	İşlem	Redux
`veriyiKaydet()`	Arıza ekle	addDoc + setArizalar
`arizalariYukle(işletmeId)`	Tüm arızaları getir	getDocs + filter
`guncelleAriza()`	Arızayı tamamla	updateDoc

Modül 2: BAKIM (Bakım Yönetimi)

Amaç

Arızaları takip etmek, tamamlamak ve geçmiş bakım kayıtlarını görmek.

Ana Ekran: `bakim` (Bakım Menüsü)

```
BAKIM MENÜ
├─ [AÇIK BİLDİRİMLER]     → setCurrentScreen('bakimAktif')
│  └─ Makine: _____  Arıza: Titreme
├─ [GEÇMİŞ ARIZALAR]      → setCurrentScreen('bakimGecmis')
├─ [YILLIK BAKIM]         → setCurrentScreen('bakimYillikAna')
├─ [HAFTALIK BAKIM]       → setCurrentScreen('bakimHaftalikAna')
├─ [BİNA BAKIM]           → setCurrentScreen('bakimBina') [Not impl.]
└─ [YARDIMCI İŞLETMELER]  → setCurrentScreen('bakimYardimci') [Not impl.]
```

Alt Ekran 1: `bakimAktif` (Açık Bildirimler)

Bileşen	Detaylar	Akış
Liste	Arızalar filtered `status='bildirildi'`	GET duruslar + filter
Kart Basıl	Makine, Tarih, Sipariş No göster	SELECT & load detaylar
DETAIL	guncelleAriza() formunu aç	setState(selectedAriza)

Arıza Detay Form: `bakimDetail`

Alan	Kontrol	Zorunlu	Firestore
Arıza Tarihi	Readonly	-	tarih_baslangic
Arıza Saati	Readonly	-	saat_baslangic
Vardiya	Picker (vardiyalar[])	✓	bakim_vardiya
Teknisyen	Picker (isimler[])	✓	bakim_teknisyen
Arıza Nedeni	Picker (nedenler[])	✓	bakim_neden
Müdahale Başı	TimePicker	✓	mudahale_baslangic
Açıklama	TextMultiInput	✓	bakim_aciklama

Bitiş Tarihi	DatePicker	✓	tarih_bitis
Bitiş Saati	TimePicker	✓	saat_bitis
KAYDET BUTON	**-> guncelleAriza()**	**Process**	

guncelleAriza() Logic

```
// Çağrıldığında selectedAriza.id vardır
const guncelleAriza = async () => {
  1. Doğrula: bakimTeknisyen + mudahaleBaslangic boş mu?
  2. Hesapla: responseTime = dakikaFarki(tarih_bas, saat_bas, tarih_bas, mudahale_bas)
  3. Eğer response_time < 0 → Hata ("Müdahale saati < Başlangıç saati")
  4. Oluştur updateObj = {
       bakim_vardiya, bakim_teknisyen, bakim_neden, bakim_aciklama,
       mudahale_baslangic, tarih_bitis, saat_bitis,
       status: 'tamamlandi',
       response_time
     }
  5. Query: duruslar {id: selectedAriza.id} → updateDoc(updateObj)
  6. Refresh: arizalariYukle(işletmeId)
  7. Navigate: setCurrentScreen('bakimAktif')
}
```

Alt Ekran 2: `bakimGecmis` (Tamamlanan Arızalar)

Bileşen	Veri Kaynağı	Gösterim
Liste	arizalar.filter(a => status='tamamlandi')	Makine, Tarih, Sonuç
Detay	READ-ONLY kart	Tüm alanlar görüntülü

Modül 3: HAFTALIK BAKIM (Planlı Haftalık Bakım)

Amaç

Haftanın belirli günlerinde belirli makinelere yapılacak rutin bakımı planlamak ve izlemek.

İki Koleksiyonlu Sistem

```
[haftalikBakimlar] ────────────────┐
  Plan Şeması                      │
  (Pazartesi, Salı, ... gibi)      │
                                   ├─→ DUAL WRITE PATTERN
[haftalikBakimTamamlandi] ─────────┘
  Tamamlanmış Kayıt
  (06.03.2026 tamamlandı)
```

Ekran 1: `bakimHaftalikAna` (Haftalık Bakım Ana Menü)

```
HAFTALIK BAKIM
├─ [YAPILACAK] (tab_active='yapilacak')
│  ├─ Pazartesi - Tornalama A1 [09:00-16:00]
│  ├─ Salı - Tornalama A2 [08:00-15:00]
│  └─ + ADD Plan
│
└─ [TAMAMLANMIŞ] (tab_active='tamamlanmis')
   ├─ 05.03 - Tornalama A1 (Ahmet Y.) ✓
   └─ 04.03 - Tornalama A2 (Mehmet K.) ✓
```

Ekran 2: `haftalikBakimTakvim` (Plan Takvimi)

İşlem	Fonksiyon	Sonuç
YAPILACAK Kartına Tıkla	`setSelectedHaftalikBakim(plan)` → `setCurrentScreen('haftalikBakimTamamla')`	Form aç
PLAN EKLE	`setCurrentScreen('haftalikBakimPlanlama')`	Input form
SİL	`haftalikBakimSil(bakimId, makineAdi)`	Sil + cascade delete

Ekran 3: `haftalikBakimPlanlama` (Yeni Plan Oluştur)

Alan	Kontrolü	Zorunlu	Firestore
Makine	Picker (makineler[])	✓	makine_adi
Hafta Günü	Picker (0-6, Pazartesi...)	✓	gun (int), gun_adi (str)
Başlangıç Saati	TimePicker	✓	baslangic_saat
Bitiş Saati	TimePicker	✓	bitis_saat

haftalikBakimEkle() Logic

```
const haftalikBakimEkle = async () => {
  1. Doğrula: planlamaHbMakine, planlamaHbGun, planlamaHbBaslangic, planlamaHbBitis
  2. Oluştur bakimData = {
       makine_adi: planlamaHbMakine
       gun: parseInt(planlamaHbGun),
       gun_adi: gunAdlari[parseInt(planlamaHbGun)],  // "Pazartesi" vb.
       baslangic_saat: planlamaHbBaslangic,
       bitis_saat: planlamaHbBitis,
       işletme_id: işletmeId,
       olusturma_tarihi: formatCurrentDateTr(),
       aktif: true,
       status: undefined (Planlama aşaması)
     }
  3. Demo MI?
     ├─ EVET: State'e ekle
     └─ HAYIR: addDoc('haftalikBakimlar', bakimData)
```

```
  4. Refresh: haftalikBakimYukle(işletmeId)
  5. Reset Form & Navigate: setCurrentScreen('bakimHaftalikAna')
}
```

Ekran 4: `haftalikBakimTamamla` (Bakım GerçekleStirme)

Alan	Kontrolü	Zorunlu	Firestore
Makine (RO)	Readonly	-	makine_adi
Günü (RO)	Readonly	-	gun_tarihi
Start Time (RO)	Readonly	-	baslangic_saat
End Time (RO)	Readonly	-	bitis_saat
Tamamlama Saati	TimePicker	✓	tamamlama_saati
Açıklama	TextMultiInput	✓	aciklama
"Çalışma Yapıldı"	Checkbox	✓	haftalikYapildi

haftalikBakimTamamla() Logic

```
const haftalikBakimTamamla = async () => {
  1. Doğrula: hbMakineAdi + hbAciklama + haftalikYapildi (checkbox)
  2. Oluştur bakimData = {
       makine_adi: hbMakineAdi,
       gun_tarihi: hbGunTarihi,
       baslangic_saat: hbBaslangicSaat,
       bitis_saat: hbBitisSaat,
       tamamlama_saati: hbTamamlamaSaati,
       aciklama: hbAciklama,
       yapan_kisi: user?.email,
       işletme_id: işletmeId,
     }
  3. Demo MI?
     ├─ EVET:
     │  ├─ addTo(haftalikBakimTamamlandi)
     │  └─ updateStatus: haftalikBakimlar[selectedId].status = 'tamamlandi'
     └─ HAYIR:
        ├─ addDoc('haftalikBakimTamamlandi', bakimData)
        └─ updateDoc('haftalikBakimlar', {status: 'tamamlandi'})
  4. Refresh: haftalikBakimYukle + haftalikBakimTamamlandiYukle
  5. Reset Form & Tab Switch: setHaftalikTabActive('tamamlanmis')
  6. Navigate: setCurrentScreen('bakimHaftalikAna')
}
```

Dual Write: haftalikBakimTamamla()

```
Frontend Form Input
   ↓
```

```
[KAYDET] Tıkla
   ├─ haftalikBakimTamamlandi.add({bakimData})
   │  └─ Tamamlanmış kaydını oluştur
   ├─ haftalikBakimlar.update({status: 'tamamlandi'})
   │  └─ Planı "tamamlandı" olarak işaretle
   └─ UI refresh: setHaftalikTabActive('tamamlanmis')
```

haftalikBakimSil() - Cascade Delete

```
// Bir plana ait tüm tamamlanmış kayıtları sil
await deleteDoc(haftalikBakimlar[bakimId])
await query(haftalikBakimTamamlandi)
  .where('makine_adi', '==', makineAdi)
  .where('işletme_id', '==', işletmeId)
  → deleteDoc(each)
```

Firestore Koleksiyonları

`haftalikBakimlar` (Planlar)

Alan	Tip	Açıklama	Örnek
işletme_id	String	Tenant key	"isletme_123"
makine_adi	String	Makinenin adı	"Tornalama A1"
gun	Integer	0-6 gün index	1 (Salı)
gun_adi	String	Gün ismi	"Salı"
baslangic_saat	String	HH:MM	"09:00"
bitis_saat	String	HH:MM	"16:00"
olusturma_tarihi	String	DD.MM.YYYY	"01.03.2026"
aktif	Boolean	Plan aktif mi	true
status	String	'aktif' | 'tamamlandi'	"aktif"

`haftalikBakimTamamlandi` (Tamamlanmış Kayıtlar)

Alan	Tip	Açıklama	Örnek
işletme_id	String	Tenant key	"isletme_123"
makine_adi	String	Makinenin adı	"Tornalama A1"
gun_tarihi	String	YYYY-MM-DD	"2026-03-05"
baslangic_saat	String	HH:MM	"09:00"
bitis_saat	String	HH:MM	"16:00"
tamamlama_saati	String	HH:MM	"15:45"

aciklama	String	Çalışma detayları	"Yağlama yapıldı"
yapan_kisi	String	Personel email	"ahmet@company.com"

Modül 4: YILLIK BAKIM (Planlı Yıllık Bakım)

Amaç

Yılda belirli haftalarda yapılacak kapsamlı bakımı planlamak ve izlemek. (Haftalık ile benzer).

Ekran 1: `bakimYillikAna` (Yıllık Bakım Ana Menü)

```
YILLIK BAKIM
├─ [YAPILACAK] (tab_active='yapilacak')
│  ├─ 2026 - Tornalama A1 (W 10-14)
│  ├─ 2026 - Tornalama A2 (W 15-18)
│  └─ + ADD Plan
│
└─ [TAMAMLANMIŞ] (tab_active='tamamlanmis')
   ├─ 2025 Tornalama A1 (W 52, Ahmet Y.) ✓
   └─ 2025 Tornalama A2 (W 50, Mehmet K.) ✓
```

Ekran 2: `yillikBakimPlanlama` (Yeni Yıllık Plan)

Alan	Kontrolü	Zorunlu	Firestore
Makine	Picker	✓	makine_adi
Yıl	Picker (2026, 2027...)	✓	yil
Başlangıç Haftası	Number (1-52)	✓	baslangic_hafta
Bitiş Haftası	Number (1-52)	✓	bitis_hafta

yillikBakimEkle() Logic

```
const yillikBakimEkle = async () => {
  1. Doğrula: tüm alanlar + baslangic_hafta ≤ bitis_hafta
  2. Validate: 1 ≤ hafta ≤ 52
  3. Oluştur bakimData = {
       makine_adi: planlamaYbMakine,
       yil: parseInt(planlamaYbYil),
       baslangic_hafta: baslangicHafta,
       bitis_hafta: bitisHafta,
       işletme_id: işletmeId,
       olusturma_tarihi: formatCurrentDateTr(),
       status: 'aktif',
     }
  4. Demo MI?
     ├─ EVET: State'e ekle
     └─ HAYIR: addDoc('yillikBakimlar', bakimData)
```

```
  5. Refresh: yillikBakimYukle(işletmeId)
  6. Reset Form: setPlanlamaYbMakine(''), etc.
}
```

Ekran 3: `yillikBakimTamamla` (Bakım Gerçekleştirme)

Alan	Kontrolü	Zorunlu	Firestore
Makine (RO)	Readonly	-	makine_adi
Yıl (RO)	Readonly	-	yil
Başl. Hafta (RO)	Readonly	-	baslangic_hafta
Bitiş Hafta (RO)	Readonly	-	bitis_hafta
Tamamlama Tarihi	DatePicker	✓	tamamlama_tarihi
Tamamlama Saati	TimePicker	✓	tamamlama_saati
Açıklama	TextMultiInput	✓	aciklama
"Çalışma Yapıldı"	Checkbox	✓	yillikYapildi

yillikBakimTamamla() Logic

```
const yillikBakimTamamla = async () => {
  1. Doğrula: hbMakineAdi + yillikYapildi (checkbox) + açıklama
  2. Oluştur bakimData = {
       makine_adi: ybMakineAdi,
       yil: parseInt(ybYil),
       baslangic_hafta: parseInt(ybBaslangicHafta),
       bitis_hafta: parseInt(ybBitisHafta),
       tamamlama_tarihi: ybTamamlamaTarihi,
       tamamlama_saati: ybTamamlamaSaati,
       aciklama: ybAciklama,
       yapan_kisi: user?.email,
       işletme_id: işletmeId,
     }
  3. Demo MI?
     ├─ EVET:
     │  ├─ addTo(yillikBakimTamamlandi)
     │  └─ updateStatus: yillikBakimlar[selectedId].status = 'tamamlandi'
     └─ HAYIR:
        ├─ addDoc('yillikBakimTamamlandi', bakimData)
        └─ updateDoc('yillikBakimlar', {status: 'tamamlandi'})
  4. Refresh: yillikBakimYukle + yillikBakimTamamlandiYukle
  5. Tab Switch: setYillikTabActive('tamamlanmis')
  6. Navigate: setCurrentScreen('bakimYillikAna')
}
```

yillikBakimSil() - Cascade Delete

```
// Bir plana ait tüm tamamlanmış kayıtları sil
await deleteDoc(yillikBakimlar[bakimId])
await query(yillikBakimTamamlandi)
  .where('makine_adi', '==', makineAdi)
  .where('yil', '==', yil)
  .where('baslangic_hafta', '==', baslangicHafta)
  .where('bitis_hafta', '==', bitisHafta)
  .where('işletme_id', '==', işletmeId)
  → deleteDoc(each)
```

Firestore Koleksiyonları

`yillikBakimlar` (Planlar)

Alan	Tip	Açıklama	Örnek
işletme_id	String	Tenant key	"isletme_123"
makine_adi	String	Makinenin adı	"Tornalama A1"
yil	Integer	Yıl	2026
baslangic_hafta	Integer	1-52	10
bitis_hafta	Integer	1-52	14
olusturma_tarihi	String	DD.MM.YYYY	"01.03.2026"
status	String	'aktif' | 'tamamlandi'	"aktif"

`yillikBakimTamamlandi` (Tamamlanmış Kayıtlar)

Alan	Tip	Açıklama	Örnek
işletme_id	String	Tenant key	"isletme_123"
makine_adi	String	Makinenin adı	"Tornalama A1"
yil	Integer	2026	2026
baslangic_hafta	Integer	Plan başlangıç	10
bitis_hafta	Integer	Plan bitiş	14
tamamlama_tarihi	String	YYYY-MM-DD	"2026-03-15"
tamamlama_saati	String	HH:MM	"17:00"
aciklama	String	Bakım detayları	"Komplet revizyon yapıldı"
yapan_kisi	String	Personel email	"mehmet@company.com"

Modül 5: PROJE (Proje Planlama)

Amaç

Yatırım/İyileştirme projelerini planlamak, takip etmek ve tamamlamak.

Ekran 1: `proje` (Proje Listesi)

```
PROJE MENÜ
├─ [YAPILACAK] (projeTabActive='yapilacak')
│  ├─ Proje-A (16.03.2026 bitiş)
│  ├─ Proje-B (30.03.2026 bitiş)
│  └─ + YENI PROJE
│
├─ [TAMAMLANAN] (projeTabActive='tamamlanmis')
│  ├─ Proje-2025-A ✓
│  └─ Proje-2025-B ✓
│
└─ [ONAY BEKLİYEN] (projeTabActive='onay_bekliyor')
   ├─ Proje-C (Onaylayan: Müdür)
   └─ Proje-D (Onaylayan: Müdür)
```

Ekran 2: `planlamaProje` (Yeni Proje Oluştur)

Alan	Kontrolü	Zorunlu	Firestore
Proje Adı	TextInput	✓	proje_adi
Başlangıç Tarihi	DatePicker	✓	baslangic_tarihi
Bitiş Tarihi	DatePicker	✓	bitis_tarihi
Proje Yöneticisi	Picker (isimler[])	✓	proje_yoneticisi
Amaç	TextMultiInput	✓	amac
Kapsam	TextMultiInput	-	kapsam
EKLE BUTON	**-> projeEkle()**	Works	

projeEkle() Logic

```
const projeEkle = async () => {
  1. Doğrula: projeAdi + bitis + yonetici + amac
  2. Oluştur projeData = {
       proje_adi: planlamaPProjeAdi,
       baslangic_tarihi: planlamaPBaslangic,
       bitis_tarihi: planlamaPBitis,
       proje_yoneticisi: planlamaPYoneticisi,
       amac: planlamaPAmac,
       kapsam: planlamaPKapsam,
       işletme_id: işletmeId,
       isletme_kimligi: işletmeId,  // Fallback key
       olusturma_tarihi: formatCurrentDateTr(),
       status: 'aktif',
```

```
      durum: 'aktif',  // Fallback key
    }
  3. Demo MI?
     ├─ EVET: projeListesi'ne ekle
     └─ HAYIR: addDoc('projeler', projeData)
  4. Refresh: projeYukle(işletmeId)
  5. Reset Form & Navigate: setCurrentScreen('proje')
}
```

Ekran 3: `projeTamamla` (Projeyi Tamamla)

Alan	Kontrolü	Zorunlu	Firestore
Proje Adı (RO)	Readonly	-	proje_adi
Başlangıç Tarihi (RO)	Readonly	-	baslangic_tarihi
Termin Tarihi (RO)	Readonly	-	bitis_tarihi (termin)
Gerçek Bitiş Tarihi	DatePicker	✓	gercek_bitis_tarihi
Sonuç Notu	TextMultiInput	✓	sonuc_notu
"Tamamlandı" Checkbox	Checkbox	✓	projeTamamlandı
TAMAMLA BUTON	**-> projeTamamlayaGonder()**	Submit	

projeTamamlayaGonder() - Dual Write Pattern

```
const projeTamamlayaGonder = async () => {
  1. Doğrula: pProjeAdi + pSonucNotu + projeTamamlandı (checkbox)
  2. Oluştur projeData = {
       ... (all fields from selectedProje)
       gercek_bitis_tarihi: pTamamlamaTarihi,
       sonuc_notu: pSonucNotu,
       yapan_kisi: user?.email,
     }
  3. Demo MI?
     ├─ EVET:
     │  ├─ addTo(projeTamamlandi)
     │  └─ updateStatus: projeler[selectedId].status = 'onay_bekliyor'
     └─ HAYIR:
        ├─ addDoc('projeTamamlandi', projeData) ←── YENI KAYIT
        └─ updateDoc('projeler', {status: 'onay_bekliyor', durum: 'onay_bekliyor'})
  4. Refresh: projeYukle() + projeTamamlandiYukle() + projeOnayBekliyorYukle()
  5. Tab Switch: setProjeTabActive('yapilacak')
  6. Navigate: setCurrentScreen('proje')
}
```

Ekran 4: `projeOnay` (Proje Onay/Reddetme)

projeOnayla(projeId) Logic

```
const projeOnayla = async (projeId) => {
  1. Demo MI?
     ├─ EVET: Eğlence mesajı
     └─ HAYIR:
        └─ updateDoc('projeler', {
             status: 'tamamlandi',
             durum: 'tamamlandi',
             onay_tarihi: Today ISO,
             onaylayan: user?.email,
           })
  2. Refresh: projeOnayBekliyorYukle() + projeYukle() + projeTamamlandiYukle()
}
```

projeReddet(projeId) Logic

```
const projeReddet = async (projeId) => {
  1. Demo MI?
     ├─ EVET: Eğlence mesajı
     └─ HAYIR:
        └─ updateDoc('projeler', {
             status: 'aktif',
             durum: 'aktif',
             red_tarihi: Today ISO,
             redjeden: user?.email,
           })
  2. Refresh: projeOnayBekliyorYukle() + projeYukle()
}
```

Proje Durum Akış Şeması

```
[Aktif] ──────────────────────────┐
 (Proje başladı, çalışılıyor)     │
        ↓                         │
[Tamamlandı (Draft)]              │
 (Proje bittiğini bildir)         │
        ↓                         │
[Onay Bekliyor]                   │
 (Yönetici kontrol)               │
        ├─ [Onaylama] ──→ [Tamamlandı (Final)]
        └─ [Reddetme] ──→ [Aktif] ──────────────┘
```

Proje Koleksiyonları

`projeler` **(Proje Planları)**

Alan	Tip	Açıklama	Örnek
işletme_id	String	Tenant key	"isletme_123"
proje_adi	String	Proje adı	"LED Aydınlatma Upgrade"

baslangic_tarihi	String	YYYY-MM-DD	"2026-03-01"
bitis_tarihi	String	YYYY-MM-DD	"2026-03-30"
proje_yoneticisi	String	Sorumlu kişi	"Erhan K."
amac	String	Proje amacı	"Enerji tasarrufu"
kapsam	String	Detaylar	"Tüm üretim alanı"
status	String	'aktif' | 'onay_bekliyor' | 'tamamlandi'	"aktif"
durum	String	Fallback key	"aktif"
olusturma_tarihi	String	DD.MM.YYYY	"01.03.2026"
isletme_kimligi	String	Fallback tenant	"isletme_123"

`projeTamamlandi` (Tamamlanan Projeler)

Alan	Tip	Açıklama	Örnek
işletme_id	String	Tenant key	"isletme_123"
proje_adi	String	Proje adı	"LED Aydınlatma Upgrade"
baslangic_tarihi	String	YYYY-MM-DD	"2026-03-01"
termin_tarihi	String	YYYY-MM-DD	"2026-03-30"
gercek_bitis_tarihi	String	YYYY-MM-DD	"2026-03-28"
proje_yoneticisi	String	Sorumlu kişi	"Erhan K."
amac	String	Amaç	"Enerji tasarrufu"
kapsam	String	Kapsam	"Tüm üretim alanı"
sonuc_notu	String	Fiilî sonuç	"Başarıyla tamamlandı, bütçe içinde"
yapan_kisi	String	Tamamlayan email	"erhan@company.com"
onay_tarihi	String	YYYY-MM-DD	"2026-03-29"
onaylayan	String	Onaylayan email	"mudur@company.com"
olusturma_tarihi	String	DD.MM.YYYY	"01.03.2026"

`projeButce` (Proje Bütçeleri - İsteğe Bağlı)

Alan	Tip	Açıklama
proje_id	String	projeler kaydının id'si
işletme_id	String	Tenant key
bilyer	String	Maliyet kalemi

miktar	Number	Miktar
birim_fiyat	Number	TL

`projeRisk` (Proje Riskleri - İsteğe Bağlı)

Alan	Tip	Açıklama
proje_id	String	projeler kayıdının id'si
işletme_id	String	Tenant key
risk_aciklama	String	Risk açıklaması
olasilik	String	'Düşük' | 'Orta' | 'Yüksek'
etki	String	'Düşük' | 'Orta' | 'Yüksek'
b_plani	String	B planı (contingency)

Modül 6: PLANLAMA (Operasyonel Planlama)

Amaç

Tüm modüllerin (Haftalık, Yıllık, Proje) planlarını toplu olarak yönetmek ve ayarları düzenlemek.

Ana Ekran: `planlama` (Planlama Menüsü)

```
PLANLAMA MENÜ
├─ [HAFTALIK BAKIM PLANLAMA] → setCurrentScreen('haftalikBakimPlanlama')
├─ [YILLIK BAKIM PLANLAMA]   → setCurrentScreen('yillikBakimPlanlama')
├─ [PROJE PLANLAMA]          → setCurrentScreen('planlamaProje')
├─ [AYARLAR]                 → setCurrentScreen('ayarlar')
│  ├─ Makineler (+ / -)
│  ├─ İsimler (+ / -)
│  ├─ Vardiyalar (+ / -)
│  └─ Nedenler (+ / -)
└─ [VERİ GİRİŞİ]             → setCurrentScreen('veriGirisi')
```

Alt Ekranlar (Detaylı Olmayan)

Tüm alt ekranlar (haftalikBakimPlanlama, yillikBakimPlanlama, planlamaProje) mevcut modüllerin planlama ekranları ile aynıdır.

Ayarlar Ekranı: `ayarlar`

Kategori	İşlem	UI	Firestore
Makineler	Ekle	yeniMakine + [EKLE BUTON]	ayarlar.makineler[]
	Sil	[X] buton	deleteAt(index)
	Liste	ScrollView	map(makineler)

İsimler (Teknisyenler)	Ekle	yeniIsim + [EKLE BUTON]	ayarlar.isimler[]
	Sil	[X] buton	deleteAt(index)
	Liste	ScrollView	map(isimler)
Vardiyalar	Ekle	yeniVardiya + [EKLE BUTON]	ayarlar.vardiyalar[]
	Sil	[X] buton	deleteAt(index)
	Liste	ScrollView	map(vardiyalar)
Nedenler (Arıza Nedenleri)	Ekle	yeniNeden + [EKLE BUTON]	ayarlar.nedenler[]
	Sil	[X] buton	deleteAt(index)
	Liste	ScrollView	map(nedenler)

Ayarlar Fonksiyonları

Fonksiyon	İşlem	Firestore
`makineEkle()`	yeniMakine state'e ekle	MANUAL updateDoc
`makineSil(index)`	Filter out	MANUAL updateDoc
`ayarlariYukle(işletmeId)`	Firestore'dan oku	getDocs('ayarlar')
`ayarlariKaydet()`	Tüm listeyi yaz	updateDoc('ayarlar')

Ayarlar Koleksiyonu

`ayarlar`

Alan	Tip	Açıklama	Örnek
işletme_id	String	Tenant key	"isletme_123"
makineler	Array	Makine isimleri	["Tornalama A1", "Tornalama A2", ...]
isimler	Array	Teknisyen isimleri	["Ahmet Y.", "Mehmet K.", ...]
vardiyalar	Array	Vardiya türleri	["Sabah", "Öğleden Sonra", "Gece"]
nedenler	Array	Arıza nedenleri	["Mekanik", "Elektrik", "Operatör"]

Firestore Veri Mimarisi

Koleksiyonlar Özeti

```
Firestore Database
│
├─ [ayarlar]              ◄── Sistem Ayarları
│  └─ işletme_id (key)
│     ├─ makineler[]
```

```
│     ├─ isimler[]
│     ├─ vardiyalar[]
│     └─ nedenler[]
│
├─ [duruslar]              ◄── ÜRETİM & BAKIM
│  └─ işletme_id, makine, status (bildirildi/tamamlandi)
│
├─ [haftalikBakimlar]      ◄── HAFTALIK PLANLAMA
│  └─ işletme_id, gun (0-6), status
│
├─ [haftalikBakimTamamlandi] ◄── HAFTALIK TAMAMLANDI
│  └─ işletme_id, makine_adi, gun_tarihi
│
├─ [yillikBakimlar]        ◄── YILLIK PLANLAMA
│  └─ işletme_id, yil, hafta aralığı, status
│
├─ [yillikBakimTamamlandi] ◄── YILLIK TAMAMLANDI
│  └─ işletme_id, makine, yil, hafta
│
├─ [projeler]              ◄── PROJE PLANLAMA
│  └─ işletme_id, status (aktif/onay_bekliyor/tamamlandi)
│
├─ [projeTamamlandi]       ◄── PROJE TAMAMLANDI
│  └─ işletme_id, gercek_bitis_tarihi
│
├─ [projeButce]            ◄── OPSIYONEL: Proje Bütçeleri
│  └─ proje_id, işletme_id
│
├─ [projeRisk]             ◄── OPSIYONEL: Proje Riskleri
│  └─ proje_id, işletme_id
│
└─ [kullanicilar]          ◄── VERİ ETİKETİ: Kullanıcı Rolleri (NOT IMPLEMENTED)
   └─ email, rol, işletme_id
```

Veri Akış Diyagramı: Arıza → Tamamlama

```
┌──────────────────────────────────────────┐
│ ÜRETIM EKRANI (Frontend)                 │
│                                           │
│ [BİLDİR BUTON] ← veriyiKaydet()          │
└──────────────────────┬────────────────────┘
                       │
                       ↓
        ┌──────────────────────┐
        │ duruslar.add({       │ ← Firestore
        │  status: 'bildirildi' │
        │  ...arıza_fields     │
        │ })                   │
        └──────────────────────┘
                       │
                       ↓
```

```
┌──────────────────────────────────┐
│ BAKIM EKRANI (bakimAktif)        │
│                                   │
│ Aktif Arızalar:                   │
│ - Arıza-1 [Çöz]                   │
│ - Arıza-2 [Çöz]                   │
└──────────────────────────────────┘
                  │
                  │ [Çöz] Tıkla
                  ↓
┌──────────────────────────────────┐
│ BAKIM DETAIL (bakimDetail)        │
│ guncelleAriza() form             │
│                                   │
│ [Vardiya] [Teknisyen] [Neden] .... │
│ [KAYDET]                          │
└──────────────────────────────────┘
                  │
                  │
                  ↓
      ┌──────────────────────┐
      │ duruslar.update({    │ ← Firestore
      │  status: 'tamamlandi' │
      │  bakim_teknisyen     │
      │  response_time       │
      │ })                   │
      └──────────────────────┘
                  │
                  ↓
┌──────────────────────────────────┐
│ BAKIM EKRANI (bakimGecmis)        │
│ Tamamlanan Arızalar:              │
│ - Arıza-1 ✓                       │
│ - Arıza-2 ✓                       │
└──────────────────────────────────┘
```

Multi-Tenant Mimarisi

Tenant Isolation Pattern

Tüm koleksiyonlarda **işletme_id** kullanılarak çok kiracılı ortam sAglanmaktadır.

Login & İşletme ID Atanması

```
onAuthStateChanged(auth, async (currentUser) => {
  if (currentUser) {
    setUser(currentUser);

    // İşletme ID'sini kullanıcı özelliklerinden al
    // veya müşteri no aracılığıyla eşle
    const işletmeIdAtanmis = await getUserIsletmeId(currentUser.email);
    setİşletmeId(işletmeIdAtanmis);
```

```
    // Tüm veriler bu işletme_id ile filtrelenir
    await ayarlariYukle(işletmeIdAtanmis);
    await arizalariYukle(işletmeIdAtanmis);
    setCurrentScreen('menu');
  }
});
```

Veri Ekleme Sırasında İşletme_ID

```
// Örnek: Arıza Bildirme
await addDoc(collection(db, 'duruslar'), {
  işletme_id: işletmeId,  // ←── KRİTİK: Tenant key
  makine_adi: selectedMakine,
  ...
});
```

Veri Okuma Sırasında Filtreleme

```
// Örnek: Arızaları Yükleme
const q = query(
  collection(db, 'duruslar'),
  where('işletme_id', '==', işletmeId)  // ←── KRİTİK: Where clause
);
const querySnapshot = await getDocs(q);
```

Firestore Security Rules (Opsiyonel)

```
// Firestore.rules
match /duruslar/{document=**} {
  allow read, write: if request.auth.uid != null
    && request.resource.data.işletme_id == request.auth.custom.işletmeId;
}

match /haftalikBakimlar/{document=**} {
  allow read, write: if request.auth.uid != null
    && request.resource.data.işletme_id == request.auth.custom.işletmeId;
}

// ... (her koleksiyon için benzer)
```

Demo Mode Akışı

Demo Mode Kimliklendirme

```
const handleDemoLogin = () => {
  setIsDemoMode(true);
  setUser({ uid: 'demo-user', email: 'demo@ycmms.local' });
```

```
  setUserRole('demo');
  setİşletmeId('demo');
  setMusteriNo('DEMO');

  // Demo verisi
  setMakineler(['Makine 1', 'Makine 2', 'Makine 3']);
  setIsimler(['Teknisyen 1', 'Teknisyen 2']);
  setVardiyalar(['Vardiya 1', 'Vardiya 2', 'Vardiya 3']);
  setNedenler(['Mekanik', 'Elektrik', 'Operatör']);
  setArizalar([]);
  setCurrentScreen('menu');
};
```

Demo Mode Kondisyonları (Her Fonksiyonda)

```
const veriyiKaydet = async () => {
  // ...
  try {
    if (isDemoMode) {  // ◄── KONTROL NOKTASI
      // 1. Demo versiyonu: Firestore'a yazma yok
      const demoAriza = {
        id: `demo-${Date.now()}`,
        işletme_id: 'demo',
        ... (all fields)
      };
      // 2. State'e ekle (in-memory)
      setArizalar([demoAriza, ...arizalar]);
      // 3. Uyarı
      Alert.alert('Başarılı', 'Demo kayıt oluşturuldu');
      // 4. Reset & navigate
      setCurrentScreen('menu');
      return;  // ◄── ERKEN ÇIKIŞ
    }

    // Gerçek mod: Firestore'a yaz
    await addDoc(collection(db, 'duruslar'), { ... });
    // ...
  } catch (error) {
    // ...
  }
};
```

Demo Mode Veri Taraması

Demo mode'da tüm veri **sadece state**'de tutulur:

- Sayfa yenilemesi ← Veriler kaybolur
- Oturum sonlandırma ← Veriler kaybolur
- App kapatma ← Veriler kaybolur

Amaç: Giriş yapmadan sistemin test edilmesi.

Teknik Notlar & Best Practices

1. Responsiveness Patterns

KeyboardAvoidingView (IO/Android)

```
<KeyboardAvoidingView
  behavior={Platform.OS === 'ios' ? 'padding' : 'height'}
  keyboardVerticalOffset={Platform.OS === 'ios' ? 20 : 0}
>
  {/* Form content */}
</KeyboardAvoidingView>
```

2. State Yönetimi Stratejisi

180+ useState hooks uygulamada (yoğun state):

```
// Modül bazında state grouplandırması:
// 1. Authentication state
// 2. UI state (currentScreen, tabActive)
// 3. Form state (modüle özel input fields)
// 4. Data state (koleksiyon listeleri)
// 5. Modal/Dropdown state (showDatePicker vb.)
```

3. Tarih/Saat Formatlama Fonksiyonları

```
formatCurrentDateTr()      // DD.MM.YYYY (güncel tarih)
formatCurrentDateIso()     // YYYY-MM-DD (ISO format)
formatCurrentTime5Min()    // HH:MM (5 dakika yuvarlanmış)
getWeekNumber(date)        // 1-52 (ISO hafta numarası)
dakikaFarki(...)           // İki tarih/saat arası fark (dakika)
```

4. Koleksiyon Adlandırması Kuralları

Pattern	Örnek	Doğru	Not
Plan	`haftalikBakimlar`	✓	Akış'ı temsil eder
Tamamlandı	`haftalikBakimTamamlandi`	✓	Sonuç kaydını temsil eder
İşletme Yalıtımı	`işletme_id` alanı	✓	String field (snake_case)
Durum Anahtarı	`status` veya `durum`	⚠️	Tutarlı olun (fallback var)

5. Fallback Mekanizmaları

```
// Veri okuması sırasında birden fazla alan kontrolü
const getKayitIsletmeId = (kayit) => {
  return (
```

```
    kayit?.işletme_id ||
    kayit?.isletme_id ||
    kayit?.isletme_kimligi ||
    kayit?.['işletme_kimliği'] ||
    ''
  );
};

// Status alanı için
const getKayitStatus = (kayit) => {
  return kayit?.status || kayit?.durum || '';
};
```

Neden? API değişime karşı kullanışlı; eski/yeni veri formatlarını destekler.

6. Hata Yönetimi

```
try {
  // Async işlem
  await addDoc(collection(db, 'koleksiyonAdi'), data);
} catch (error) {
  console.error('İşlem adı başarısız:', error);  // Server log
  Alert.alert('Hata', error.message);            // User facing
}
```

7. Navigasyon (Manual Routing)

App.js'te `currentScreen` state'i ile 18+ ekran koşullu render'lanır.

```
if (currentScreen === 'uretim') { return <UretimScreen /> }
if (currentScreen === 'bakim') { return <BakimScreen /> }
// ... 16 more screens
```

Risik: `setCurrentScreen('undefined_screen')` çağrısı sessiz başarısızlıkla sonuçlanır.

Tespit: Gre `setCurrentScreen('bakimProjesi')` çağrısı bulundu fakat koşullu render bloku yok → navigasyon başarısız.

8. Performance Optimizations (Önerilen)

```
// 1. useCallback hooks (şu anda yok)
const handlePress = useCallback(() => { ... }, [deps]);

// 2. useMemo lists (şu anda yok)
const activeArizalar = useMemo(() =>
  arizalar.filter(a => a.status === 'bildirildi'),
[arizalar]);
```

```
// 3. FlatList pagination (sayfalama)
// Şu anda ScrollView + map() → büyük listeler yavaş olabilir
```

9. Security Concerns (Konfigürasyon)

✓ **Firestore Security Rules** yapılandırılması önerilir
✓ **Kullanıcı roller sistem**i (rol-based access) still not fully impl.
✓ **Demo mode validation** (demo account izin listelenmeli)

Sistem Özet Tablosu

Özellik	Değer	Not
Toplam Ekran	18+	Koşullu renders
Toplam State Hooks	180+	Ağır state yönetimi
Firestore Koleksiyonları	11	Core + optional
Modüller	6	Üretim, Bakım, Haftalık, Yıllık, Proje, Planlama
İş Süreci (Bir Arıza)	4 adım	Bildir → Müdahale → Tamamla → Geçmiş
Multi-Tenant Desteği	✓	işletme_id ile isolation
Demo Mode	✓	In-memory data (persistent değil)
Authentication	Firebase Auth	Email + Şifre + Demo
Tarih/Saat	Türkçe (DD.MM.YYYY, HH:MM)	Sistem saati sync her dakika

---

Eklenti: API & Teknik Krediler

Kullanılan Kütüphaneler

- **firebase/auth** - Kimlik doğrulama
- **firebase/firestore** - Veritabanı
- **react-native-community/datetimepicker** - Tarih seçici
- **@react-native-picker/picker** - Dropdown seçici
- **react** - View, state management

Öğrenme Kaynakları

- Firebase Firestore Docs
- React Native Best Practices
- Multi-Tenancy in Firebase

Dokumen Sonu

*Güncelleme:** 6 Mart 2026
*Sorumlu:** Sistem Mimarisi Takımı
*Lisans:** İç Kullanım (YCMMS)*